

Writing sample: Project setup for Vue 3

Project brief

Project topic	Setting up Vue 3 using the Composition API
Client	DeepSource
Writer	Sara L. Santillan
Deliverable	1 article
Length	2000+ words
Includes	• Working code samples with screenshots • Github repository (for forking to client's account) • Netlify deployment (handled by client)
Time undertaken	25 hours to conceptualize, write, take screenshots, test, edit, and deploy
Project date	June 2021

The full article appears below:

VueJS is a popular front-end framework typically used to build single-page apps (SPAs) and dynamic web pages. Its architecture, which focuses on the use of components, makes it suitable for quick prototyping, and the active community support for Vue rounds out the framework with functionalities that turn prototypes into production-ready apps. This tutorial will set up a **Vue 3** project with the goal of introducing Vue 3 to both framework newcomers and past Vue 2 developers. Familiarity with NodeJS, code editors, and general web development is required for this guide.

At the end of this tutorial, you will know:

- How to set up a Vue 3 project and related tools such as VSCode and Vue Devtools

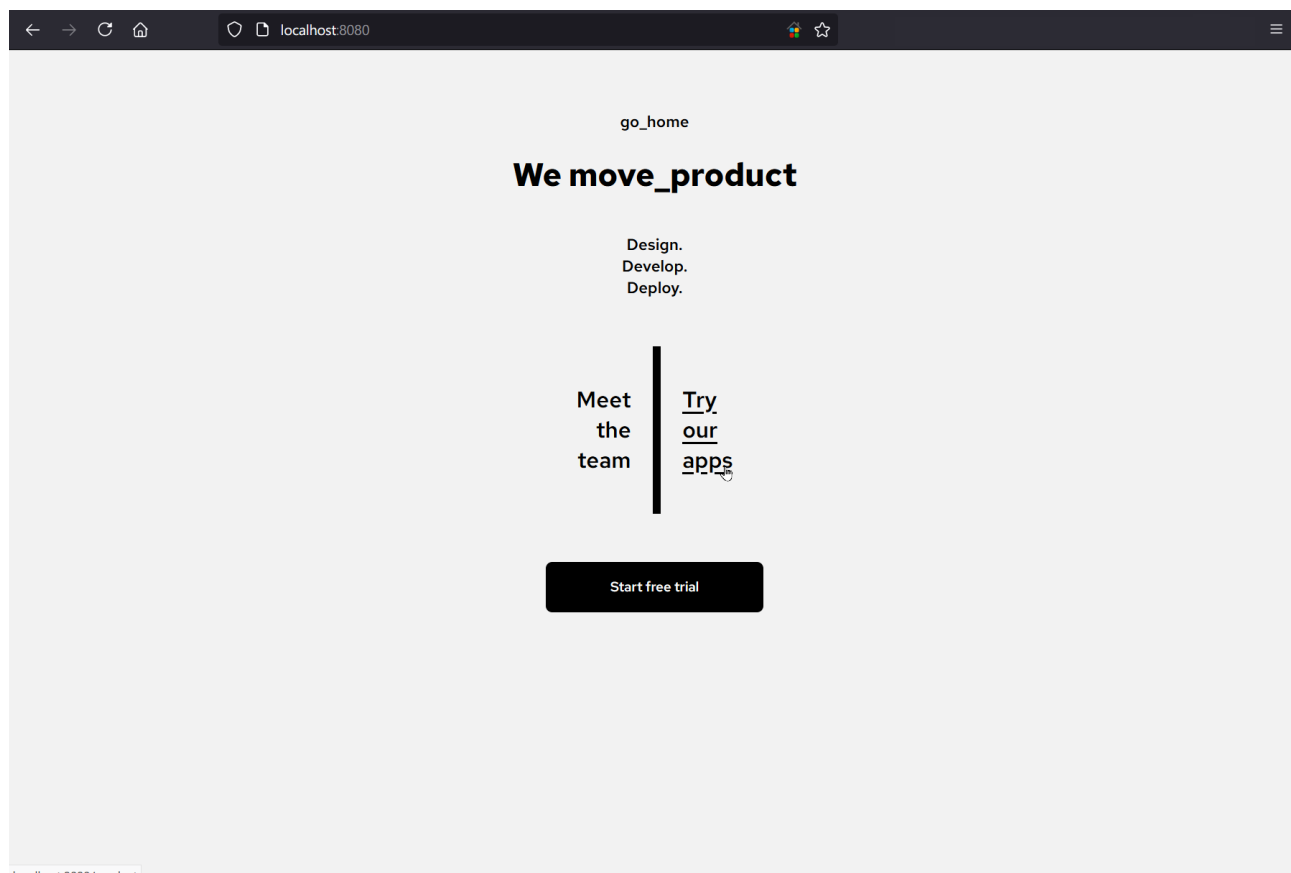
- General Vue 3 architecture: the App, Views, and Components, and how to route between Views using `vue-router`
- Component file architecture, creation, registering, rendering, and the use of `props` to feed data from parent to child
- How to use Javascript's built-in `fetch()` in Vue to consume REST APIs

The code screenshots in this guide use two optional packages to improve the readability of code samples: pug for HTML templating and stylus for CSS preprocessing. Actual code snippets are written in plain HTML/JS/CSS.

Our use case: a team member component

To illustrate Vue's components system and common operations such as routing, rendering, and API data fetching, we'll be creating a **dynamic landing page** that makes use of open-source data taken from <https://randomuser.me/> to randomly create team members in a fictitious development studio called *move_product*.** This project is deployed [here](#) for your reference.

Let's take a look at the final styled product as a goal to work towards in this guide:

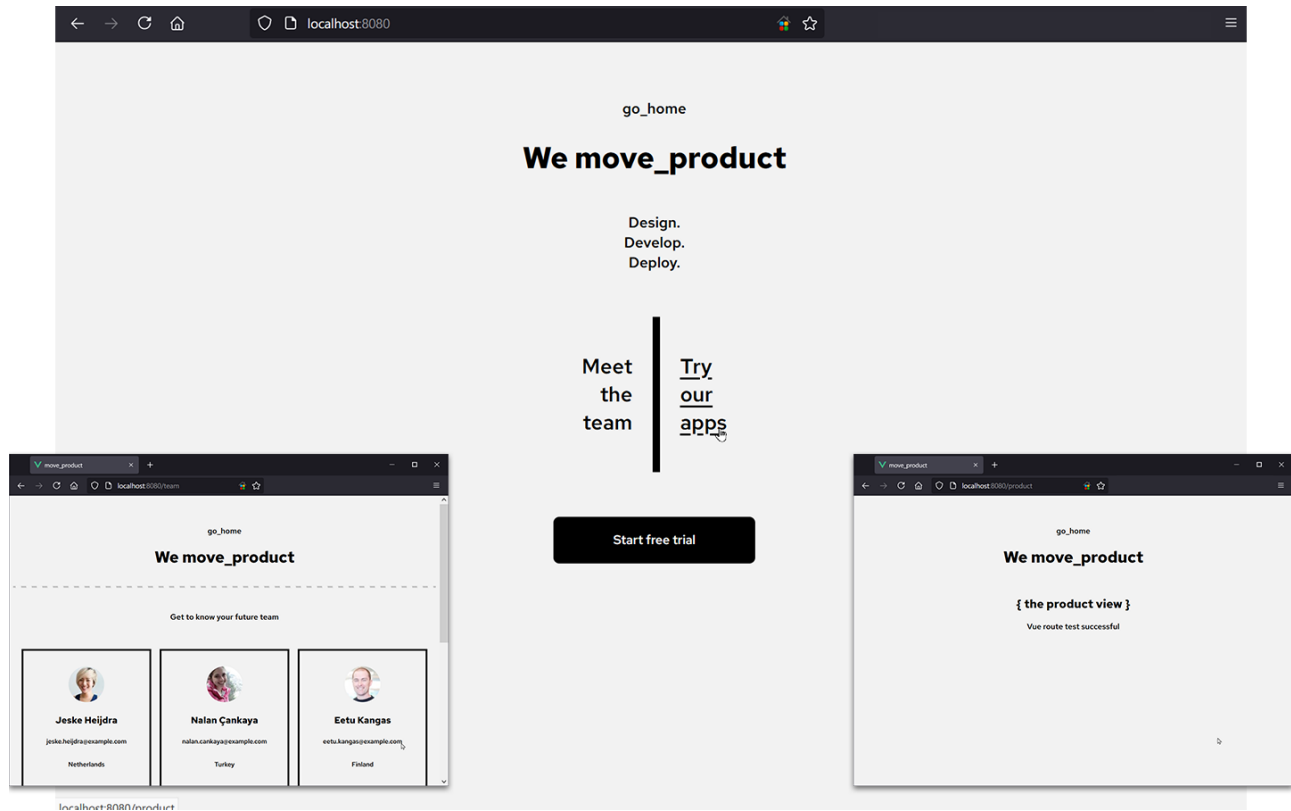


The app root (/) has two routes: `Meet the team` and `Try our apps`. `go_home` points to `/`.

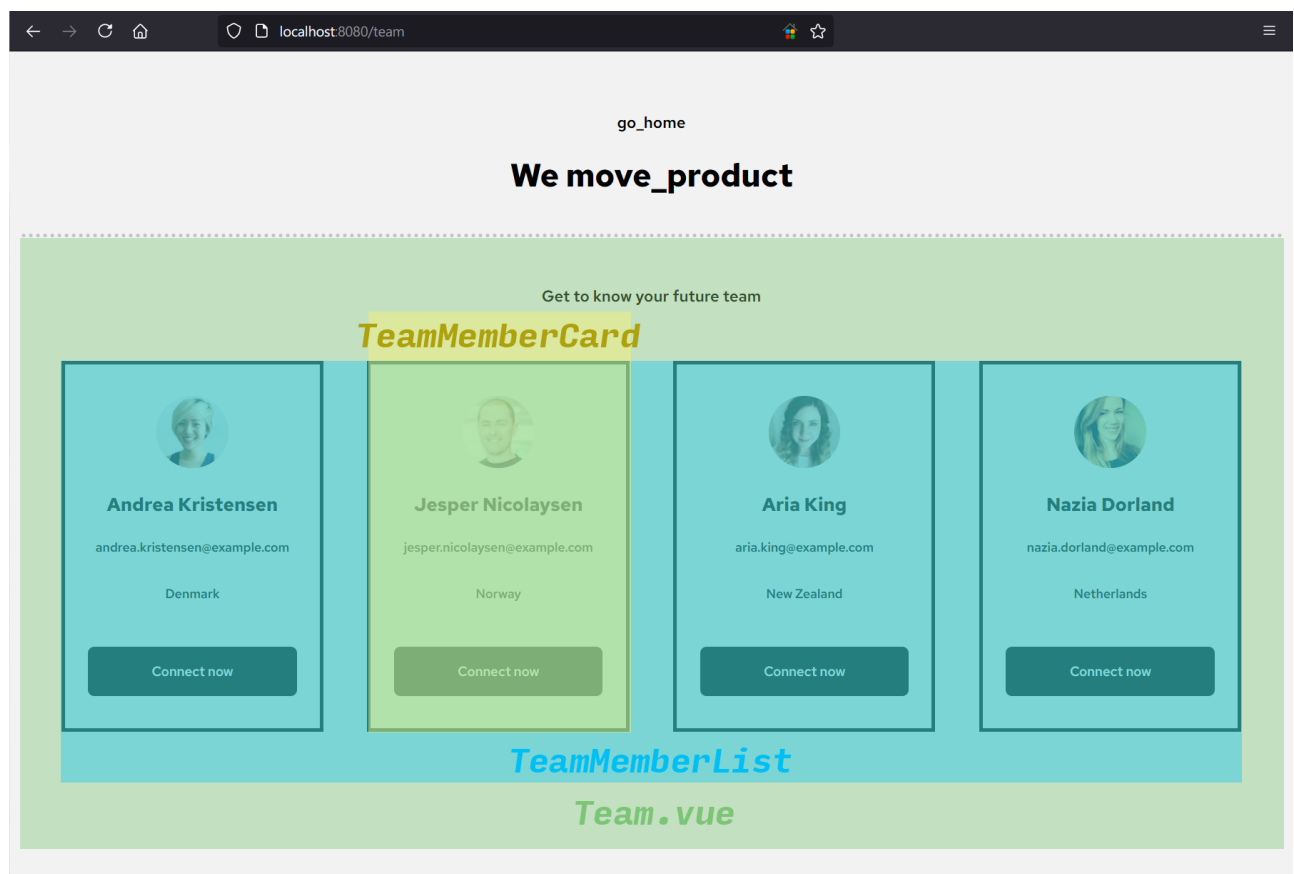
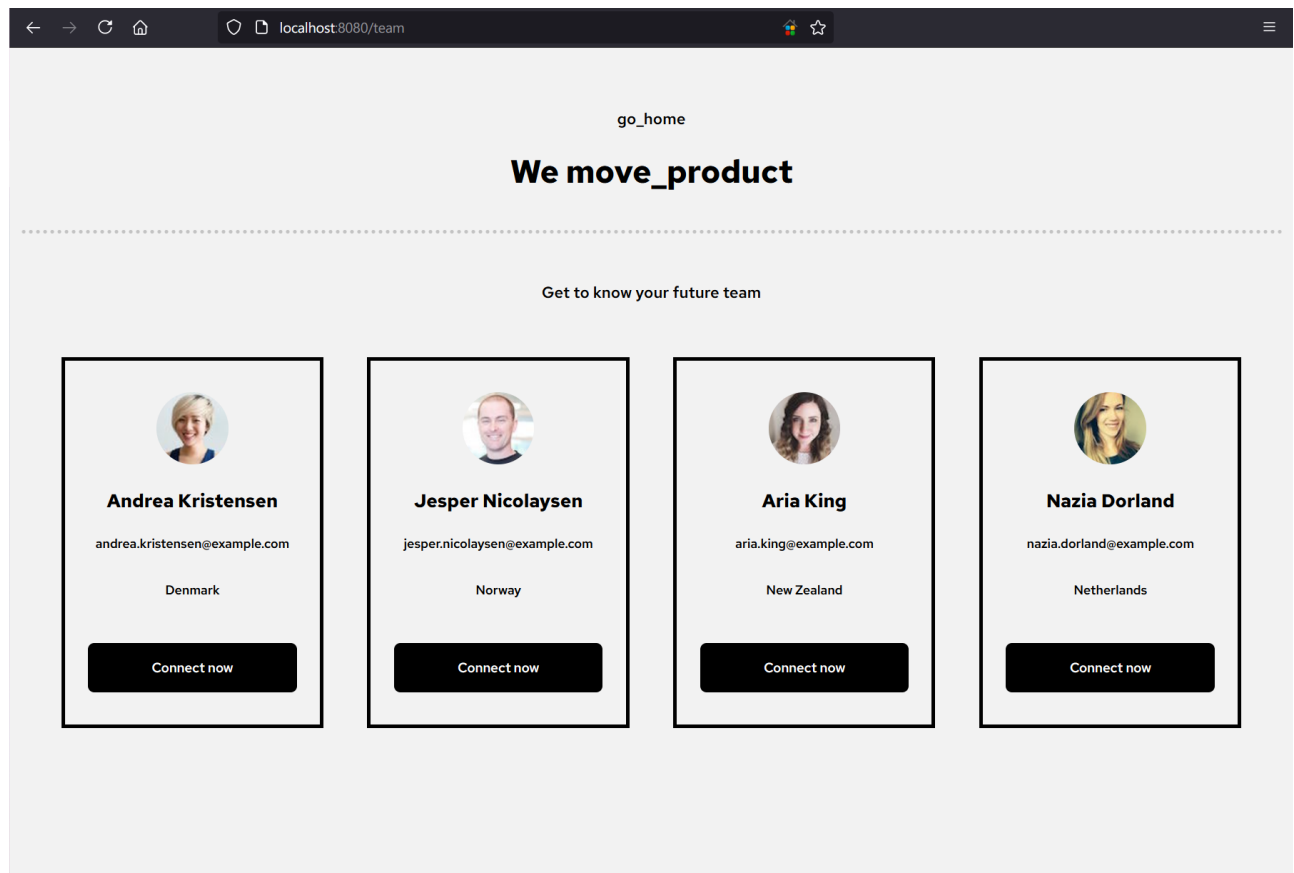
We'll set up the Views and route them with `vue-router`.

Buttons and their associated events are beyond the scope of this guide. As such, `Start free trial` is decorative and does nothing.

Here's the root with both `/team` and `/product`:



With styling and fetched data – all done in Vue files – this is what `TeamMemberList` and `TeamMemberCard`, the card components, look like within the `/team` view:



⚠ A note about CSS styling:

The goal of this guide is for readers **to feel comfortable developing with Vue**. While **CSS styles are included as we go along**, the guide will remain focused on Vue vocabulary and topics.

Creating and navigating the vue project

Vue 3

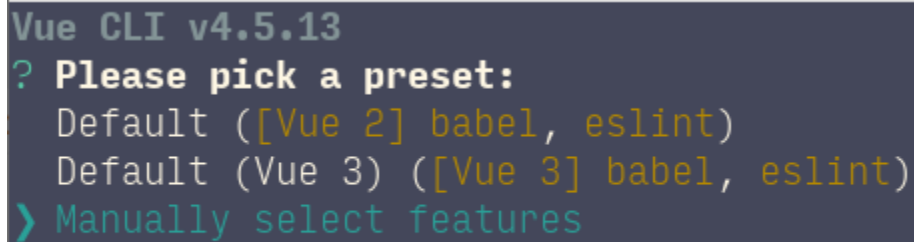
Install `vue cli` through `npm`:

```
npm update -g @vue/cli
```

Let's create our project:

```
vue create move_product
```

Let's **manually select features** to be able to set up a project with a router and additional packages.



```
Vue CLI v4.5.13
? Please pick a preset:
  Default ([Vue 2] babel, eslint)
  Default (Vue 3) ([Vue 3] babel, eslint)
> Manually select features
```

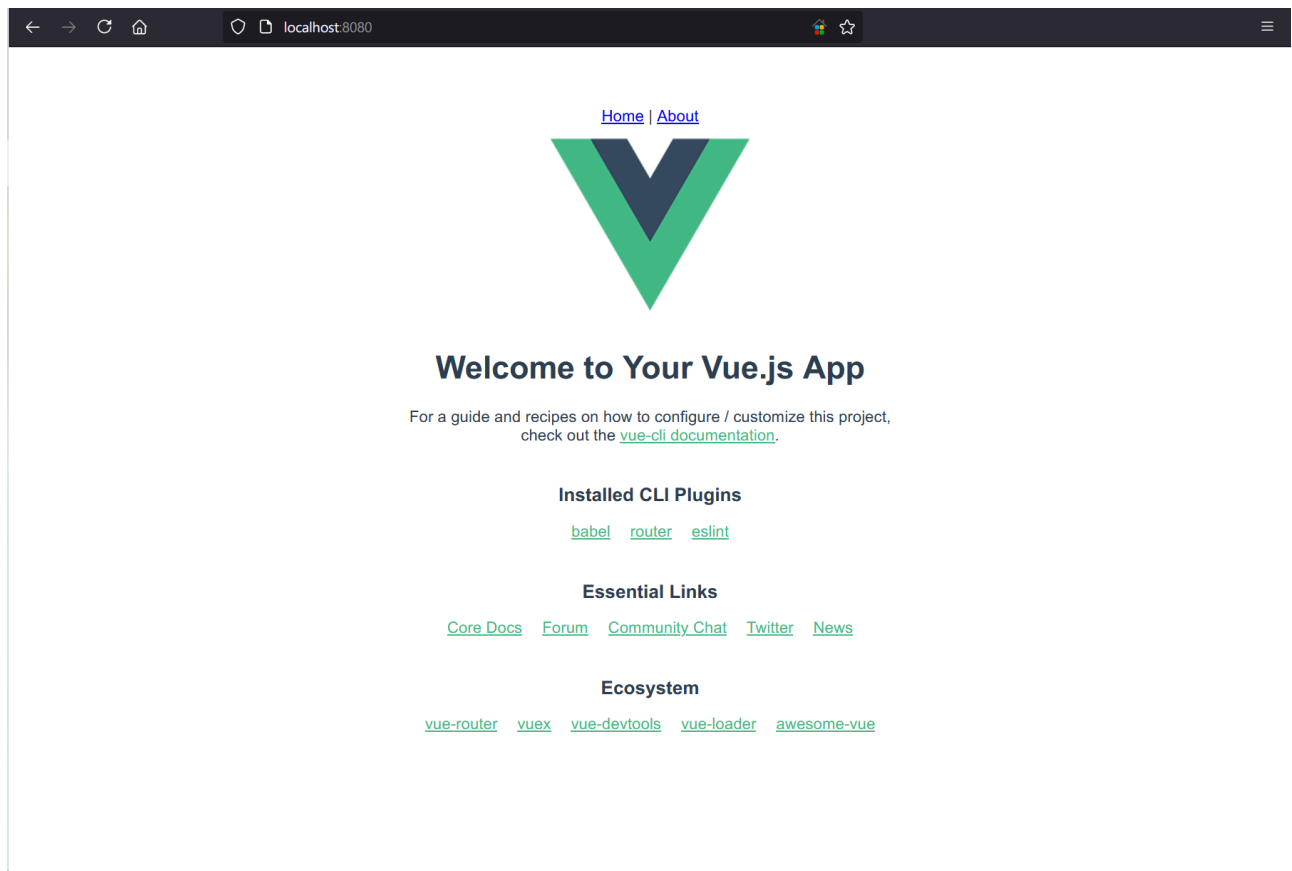
In this case, Vue will set up a project with a router, CSS preprocessor, ESLint, and Babel:

```
Vue CLI v4.5.13
? Please pick a preset: Manually select features
? Check the features needed for your project:
  ☒ Choose Vue version
  ☒ Babel
  ☐ TypeScript
  ☐ Progressive Web App (PWA) Support
  ☒ Router
  ☐ Vuex
  ☒ CSS Pre-processors
  ☒ Linter / Formatter
  ☐ Unit Testing
  ☐ E2E Testing
```

Take note of the answers for the script:

```
Vue CLI v4.5.13
? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, Router, CSS Pre-processors, Linter
? Choose a version of Vue.js that you want to start the project with 3.x
? Use history mode for router? (Requires proper server setup for index fallback in production) Yes
? Pick a CSS pre-processor (PostCSS, Autoprefixer and CSS Modules are supported by default): Stylus
? Pick a linter / formatter config: Standard
? Pick additional lint features: Lint on save
? Where do you prefer placing config for Babel, ESLint, etc.? (Use arrow keys)
  ☒ In dedicated config files
  ☐ In package.json
```

Let the tool run and you'll have your first Vue 3 app ready for development. Here's what your localhost may look like at this point:



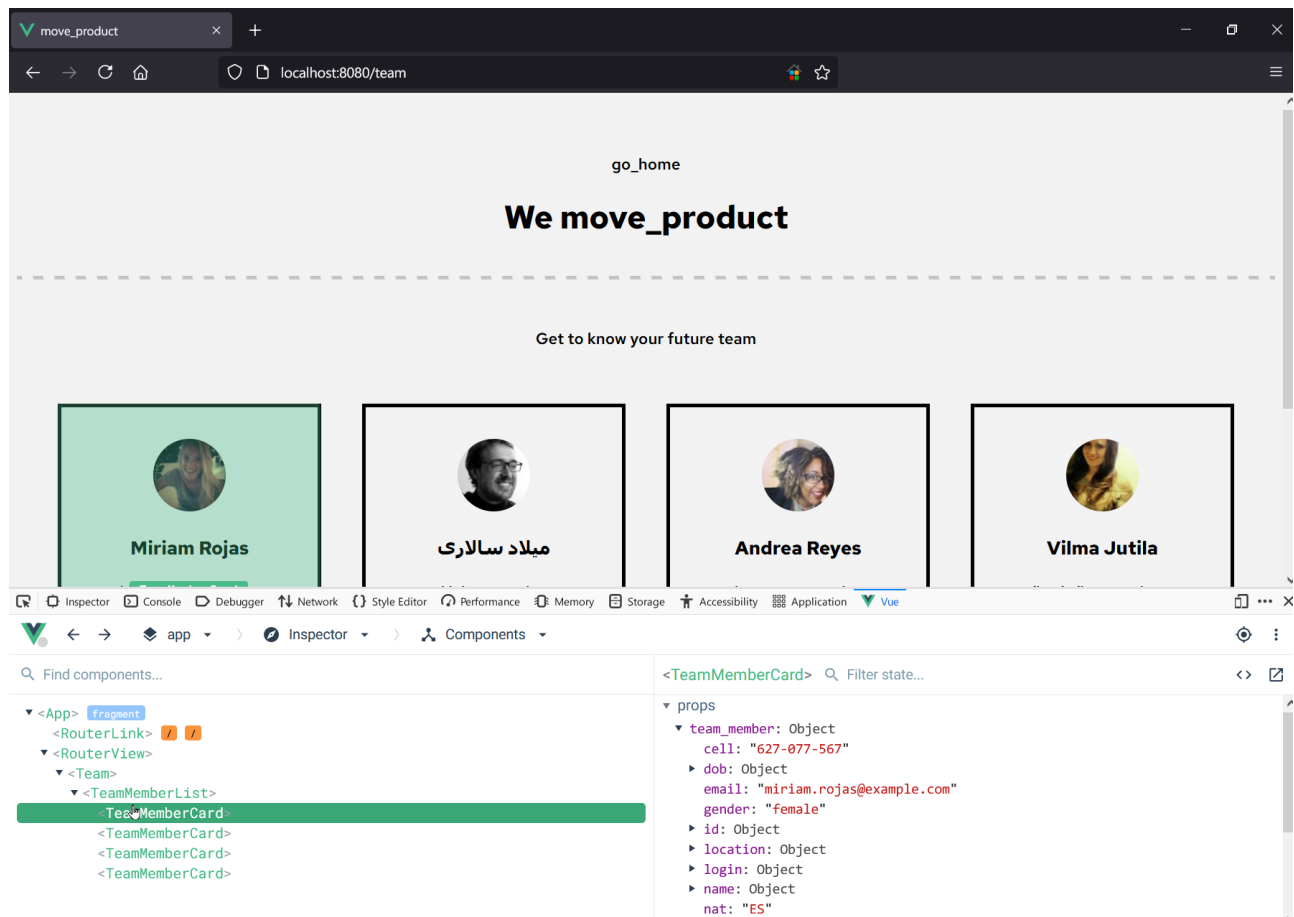
⚠ **Do I need all this?**

- **Babel** is a JS compiler to add support for older browsers.
- **ESLint** is a linter that flags coding errors, style issues, and the like.
- **stylus** is a CSS Preprocessor – these preprocessors compile to, or generate, plain CSS but are written with an easier-to-read syntax, variable definitions, the use of mixins (repeatable CSS chunks), and more.

Your feature set is up to you and the intent of the app you're developing. Additional tooling can always be added down the line.

Additional tooling: Vue Devtools 6 and Vetur+ESLint for VSCode

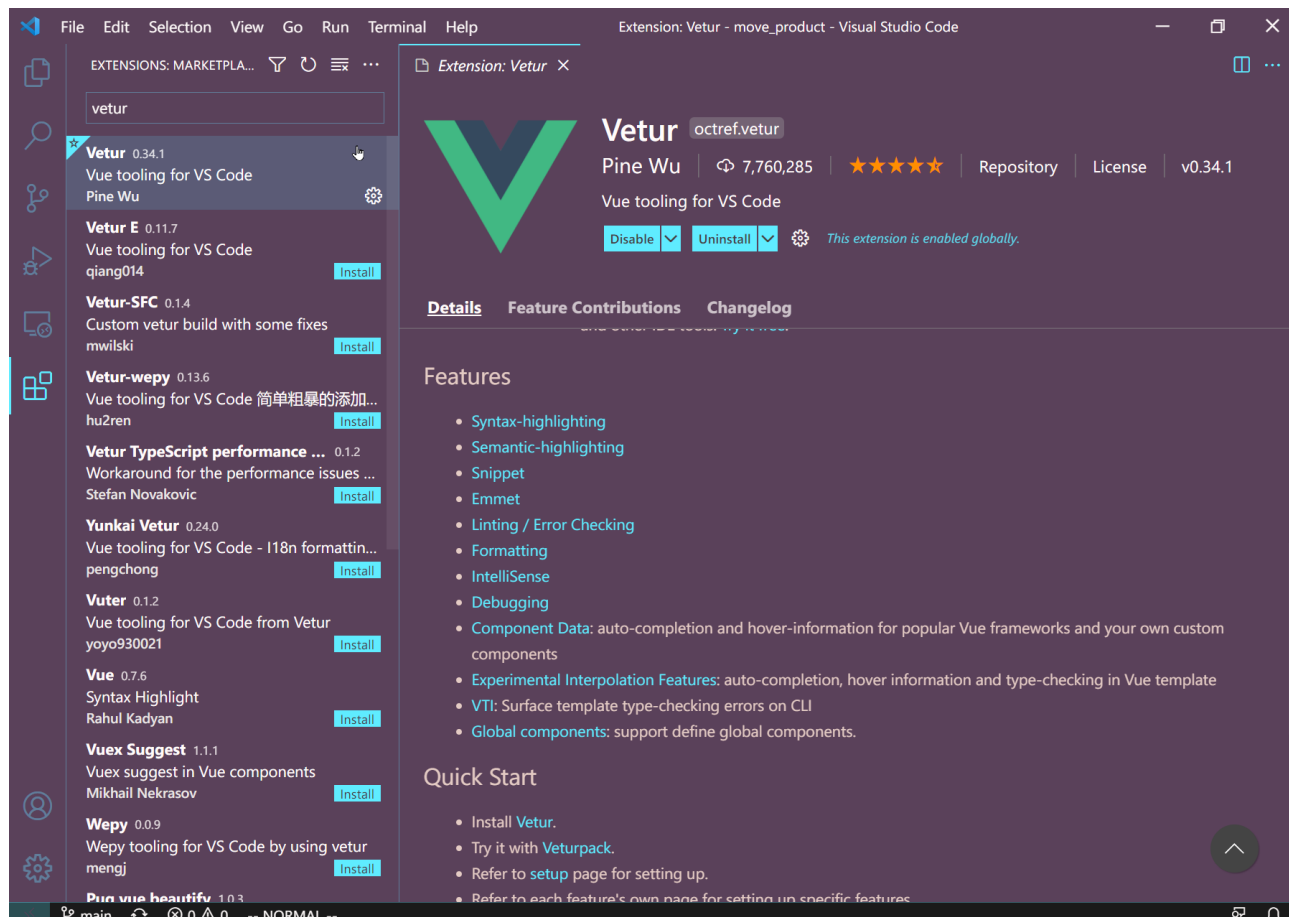
The **Vue Devtools** plugin handles the inspection of Components and Views within the browser. It is common in development to run into bugs with arrays, objects, and events – perhaps the wrong variable name was passed as a prop, or a type mismatch occurred from parent to child, or some button has failed to react to changes or fire a validation function. This is where the Devtools plugin comes in. Grab it for [Chrome](#) or for [Firefox](#). Here it is in action:



[VSCode](#) is an actively developed, actively-supported code editor – two things that determine the longevity of a tool. At the minimum, setting up the Vetur and ESLint extensions (if you've decided to use ESLint) will provide you with syntax highlighting, code snippets, and linting for JS. For this sample project, setting up `vetur` involves creating a `jsconfig.json` with the path to our Vue files:

```
{
  "include": [
    "**/src/**/*.vue"
  ]
}
```

You can download Vetur and ESLint from the extensions tab:



⚠️ A note on coding comfort and style

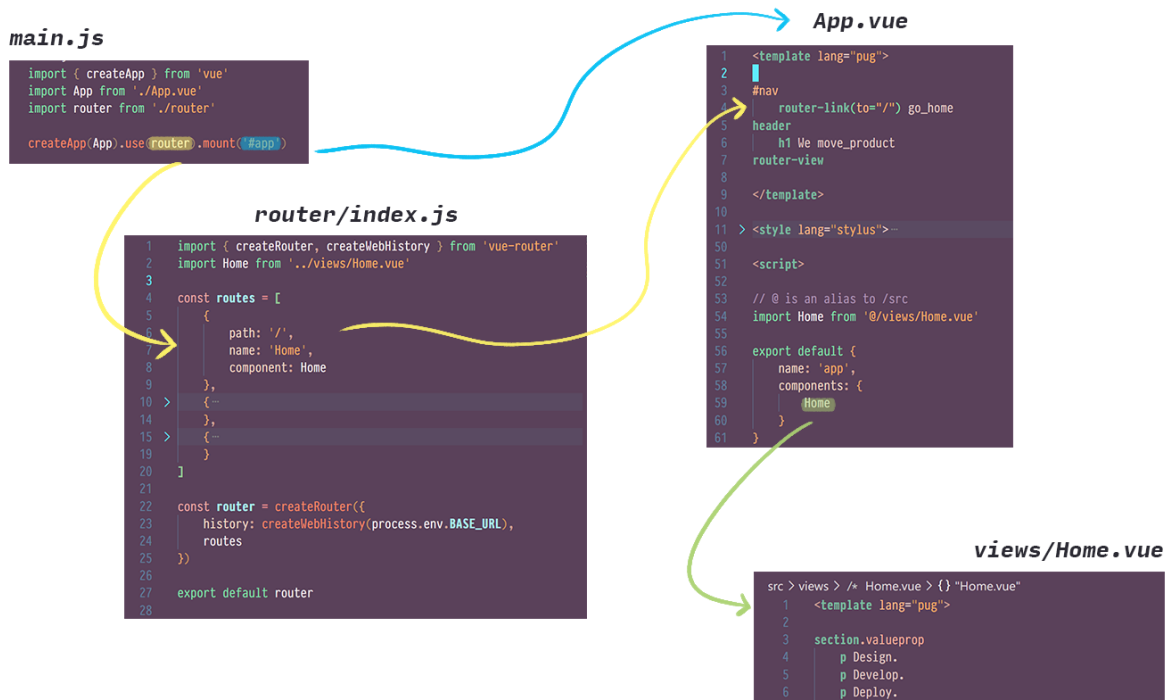
Repetitive issues such as trailing whitespace or indentation can be fixed on save in VSCode.

Open your `settings.json` file (press `Ctrl+Shift+P`, type "settings" and it'll show up on the drop-down box) and add:

```
"files.trimTrailingWhitespace": true,  
"vetur.format.options.tabSize": 4 // use your preferred value
```

Adding our routes through vue-router

Vue uses `router-link` and `router-view` directives (custom tags) to designate what components/views are to be rendered on the page. Let's look at how the `vue` object is initialized and how it pulls up the Home view:



1. `main.js` mounts the Vue App, including the router.
2. The Home View is defined in `router/index.js` and registered as a component in `App.vue`.
3. Then its content in `views/Home.vue` is rendered in `router-view` (line 7, `App.vue`)

Our goal is to simply ensure our **routes are up and running**.

1. Delete `components/HelloWorld.vue` and `views/About.vue`.
2. Create the `Team.vue` and `Product.vue` files.
3. Set up the following:

`views/Home.vue`

`<template>`

`<section class="valueprop">`

`<p>Design.</p>`

`<p>Develop.</p>`

`<p>Deploy.</p>`

`</section>`

```
<ul class="menu">
  <!-- router-link provides the anchors -->
  <li class="left"><router-link to="/team">
    Meet<br>the<br>team
  </router-link></li>
  <li class="right"><router-link to="/product">
    Try<br>our<br>apps
  </router-link></li>
</ul>
```

```
<button>Start free trial</button>
```

```
</template>
```

```
<script>
```

```
// @ is an alias to /src
// import HelloWorld from '@components/HelloWorld.vue'
```

```
export default {
  name: 'Home',
  components: {
  }
}
```

```
</script>
```

```
App.vue
```

```
<template>
```

```
<div id="nav">
  <router-link to="/">go_home</router-link>
</div>
```

```
<header>
```

```
  <h1>We move_product</h1>
```

```
</header>
```

```
<!-- router-view is where Home, Team and Product views will render -->
```

```
<router-view />
```

```
</template>
```

```
<script>
```

```
import Home from '@/views/Home.vue'
```

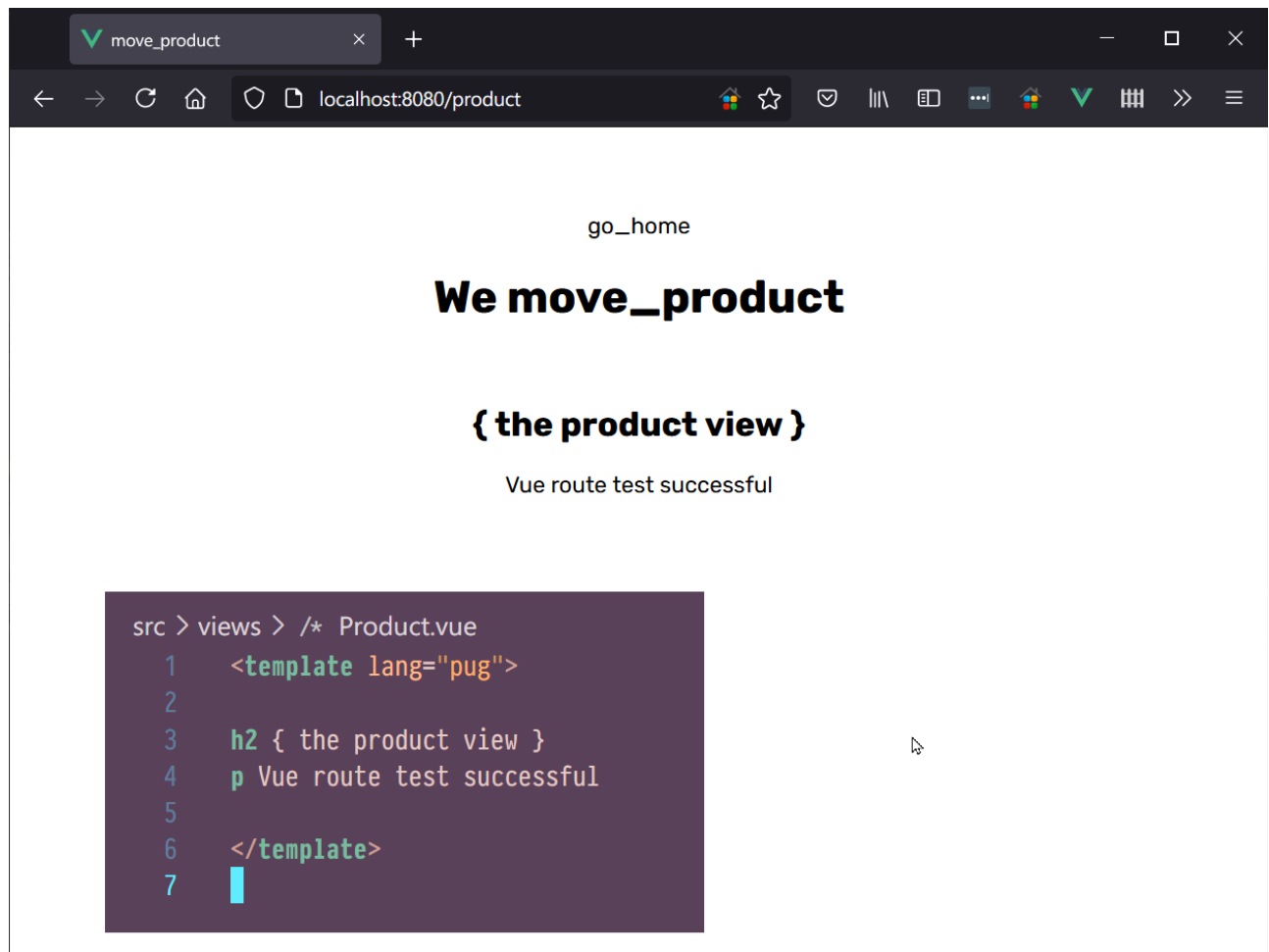
```
export default {  
  name: 'app',  
  components: {  
    Home  
  }  
}
```

```
</script>
```

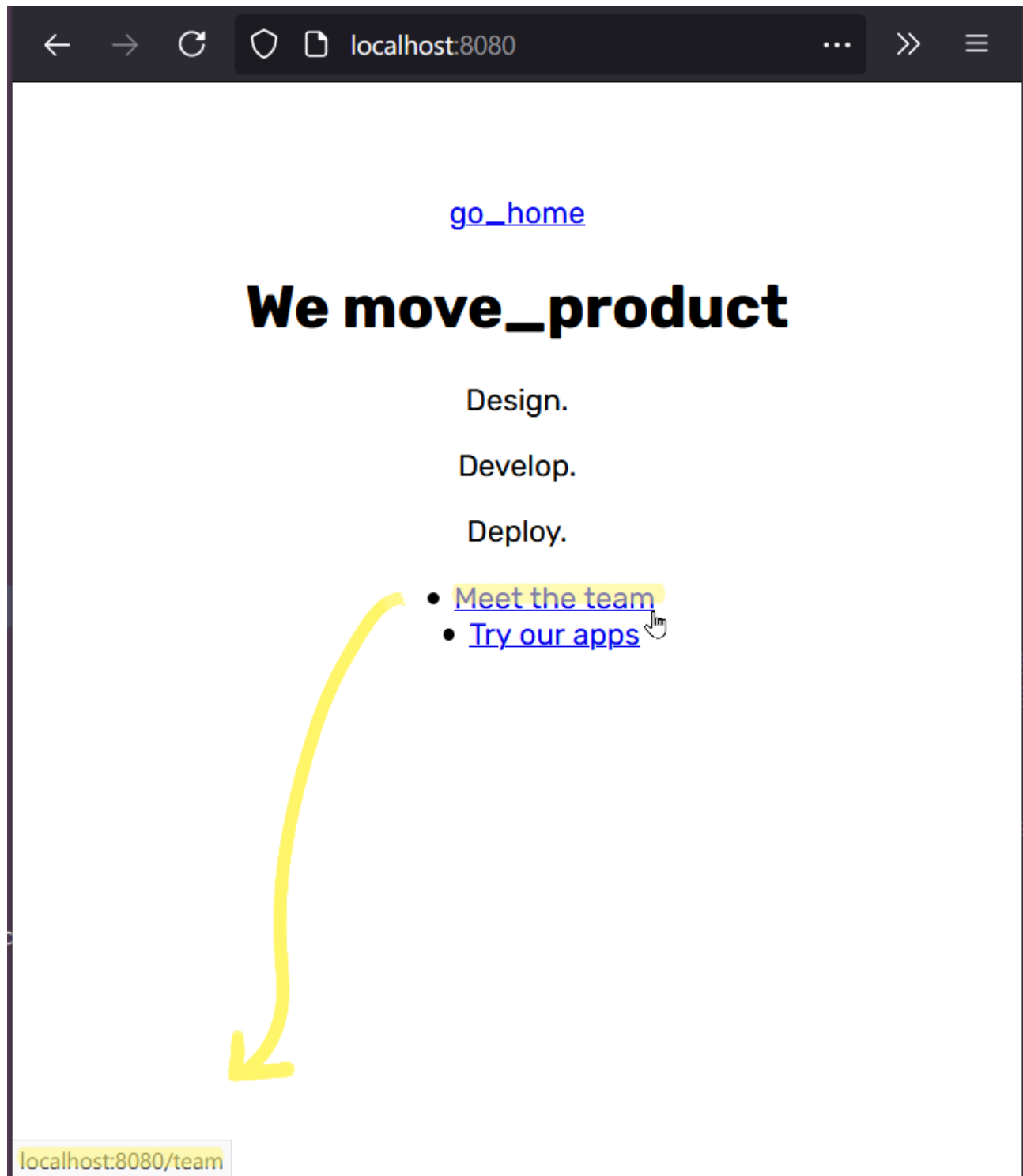
`router/index.js` starting from line 4, the routes definition block:

```
const routes = [  
  {  
    path: '/',  
    name: 'Home',  
    component: Home  
  },  
  {  
    path: '/team',  
    name: 'Team',  
    component: () => import('../views/Team.vue') // for lazy loading  
  },  
  {  
    path: '/product',  
    name: 'Product',  
    component: () => import('../views/Product.vue')  
  }  
]
```

Your `/team` and `/product` Views need not contain anything more than some text to identify that the route was successfully made, such as:



At this point, you'll have working, albeit unstyled, routes:



At this point, we can improve the page's look with CSS.

App.vue:

```
<style>
@import url('https://fonts.googleapis.com/css2?family=Red+Hat+Text:wght@500&
display=swap');
@import url('https://fonts.googleapis.com/css2?family=Red+Hat+Text:wght@700&
```

```
display=swap');

body {
  background-color: #f2f2f2;
  font-family: "Red Hat Text", sans-serif;
  font-weight: 500;
}
header {
  padding-bottom: 1rem;
}
#app {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #000000;
  margin-top: 60px;
}
a {
  text-decoration: none;
  color: #000000;
}
a:hover {
  text-decoration: underline;
}
ul {
  list-style-type: none;
  padding: 0rem;
}
button {
  padding: 1rem 4rem;
  border: none;
  border-radius: 0.4rem;
  background-color: #000000;
  color: #ffffff;
  font-family: "Red Hat Text", sans-serif;
}
</style>
```

`Home.vue` (Note the "scoped" attribute, which tells the component not to cascade the style.)

```
<style scoped>
```

```
h1 {
    font-weight: 700;
    font-size: 2.4rem;
    padding: 1rem;
}

p {
    margin: 0rem;
}

ul.menu {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    margin 3rem auto;
    padding: 0;
}

ul li {
    padding: 2.4rem 1.6rem;
    list-style: none;
    font-size: 1.4rem;
}

ul li.left {
    box-shadow: 4px 0;
    text-align: right;
}

ul li.right {
    box-shadow: -4px 0;
    text-align: left;
}

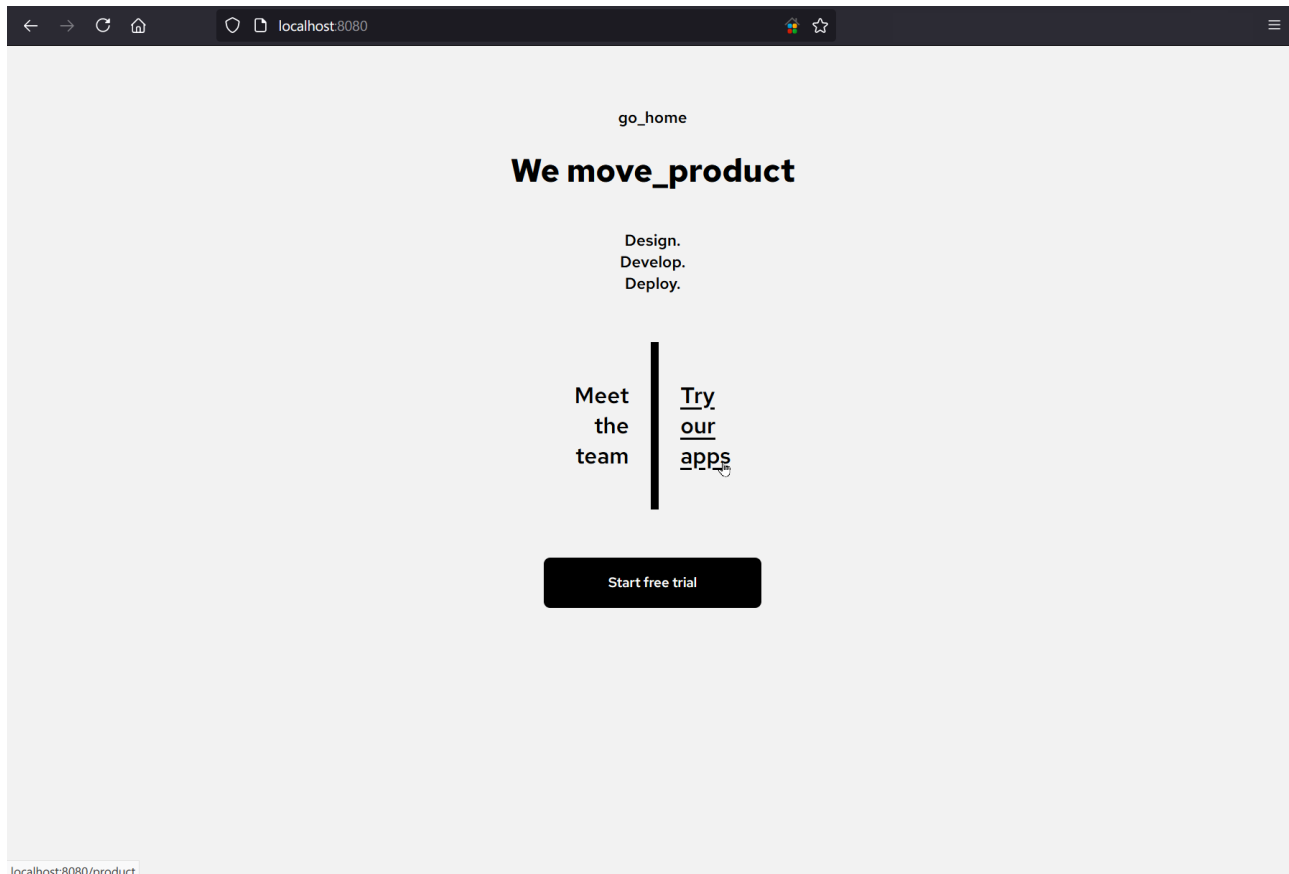
button {
    padding: 1rem 4rem;
    border: none;
    border-radius: 0.4rem;
    background-color: #000000;
    color: #ffffff;
    font-family: "Red Hat Text", sans-serif;
}

button:hover {
    cursor: pointer;
}
```

```
background-color: #303030;
}

</style>
```

Your end result should be similar to this:



Fetching with Vue 3's Composition API

Each `.vue` file returns an instance of that Vue object (whether View or Component). Behaviors such as reactivity, data passing, validation, etc. are all defined within the Vue object. Here's a sample **structure** of a `.vue` file:

```
<!-- SAMPLE STRUCTURE -->
<!-- HTML goes here: -->
<template></template>

<!-- CSS styles go here; add "scoped" to limit styles to this object only -->
<style></style>
```

```

<!-- JS goes here: -->
<script>
import [SomeObject] from [File]

export default {
  name: [ObjectName],
  props: {
    propName: {
      type: //data type such as Array or String
      required: //boolean
    }
  }
  components: {
    // register components here
    ComponentName
  },
  setup() {
    //variables (once declared in data() in Options API),
    //methods,
    //and lifecycle hooks, i.e. onMount
    return {
      // values returned from here are available
      // to the entire component
    }
  }
  //note that `this` is not used in setup()
}
</script>

```

Vue 3's **Composition API** has introduced several changes:

- the use of `ref`, which makes variables reactive (expect to see it more often)
- the use of `setup()` – where methods, lifecycle hooks, and variables now live
- the lack of `this` in `setup()` (and in all lifecycle hooks within): `setup()` is called before other options are resolved, so `this` behaves differently than the `this` object in Options (Vue 2) API.

Let's create a `fetch` function within this new structure. Fire up your `Team.vue` and update its `script` block:

```

<script>
export default {
  name: 'Team',
  components: {
    TeamMemberList //we'll be passing teamMembers to this component
  },
  setup () {
    const teamMembers = ref([])
    //teamMembers is now reactive and will be updated with its response
    async function fetchMembers () {
      const url = '<https://randomuser.me/api/?results=4>'
      await fetch(url)
        .then(response => response.json())
        .then(data => {
          teamMembers.value = data.results
        })
    }

    fetchMembers()

    return {
      teamMembers
    }
  }
}
</script>

```

This code passes the URL for `fetch()`'s GET request, then updates `teamMembers.value` with the data after the async function resolves. Recall that `teamMembers` was defined with `ref`, making it a **mutable object**; we update and access its inner values with `.value`, hence `teamMembers.value = data.results`. There are no lifecycle hooks for this simple request.

Creating a component file

The `/team` View will now receive data about `move_product`'s employees. Each employee will have their own entry. That means we have our first component: `TeamMemberCard`. We also need a component to contain those cards, which also allows us to reuse the list elsewhere in the future: `TeamMemberList`.

⚠ Components vs Views

Both Components and Views are Vue objects. The difference lies in usage and architecture: Views are explicitly routed to; whereas Components are typically parts of Views. While making a single view of a `TeamMemberCard` is possible, it's unlikely given the content and layout.

1. Create `TeamMemberCard.vue` and `TeamMemberList.vue` in the `components` folder
2. Pass `teamMembers` to the list component and then a single `teamMember` to the card component

`TeamMemberList.vue`:

```
<template>

<section>
  <ul class="team-member-list">
    <li class="team-member" v-for="(teamMember, index) in teamMembers"
:key="index" :teamMember="teamMember">
      <TeamMemberCard v-bind:teamMember="teamMember" />
    </li>
  </ul>
</template>

<script>

import TeamMemberCard from './TeamMemberCard.vue'

export default {
  name: 'TeamMemberList',
  props: {
    //TeamMemberList receives teamMembers from Team.vue
    teamMembers: {
      type: Object,
      required: true
    }
  },
  components: {
    TeamMemberCard
  }
}
```

```
</script>
```

```
TeamMemberCard.vue:
```

```
<template>
```

```
<ul class="team-member-card">
```

```
  <li>
```

```
    
```

```
  </li>
```

```
  <li>
```

```
    <h3> {{ teamMember.name.first }} {{ teamMember.name.last }}</h3>
```

```
  </li>
```

```
  <li>
```

```
    <a :href="'mailto:' + teamMember.email"> {{ teamMember.email }}</a>
```

```
  </li>
```

```
  <li>
```

```
    <p>{{ teamMember.location.country }}</p>
```

```
  </li>
```

```
  <li>
```

```
    <button>Connect now</button>
```

```
  </li>
```

```
</ul>
```

```
</template>
```

```
<script>
```

```
export default {
```

```
  name: 'TeamMemberCard',
```

```
  props: {
```

```
    //Received from TeamMemberList through v-bind directive
```

```
    teamMember: {
```

```
      type: Object,
```

```
      required: true
```

```
    }
```

```
  }
```

```
}
```

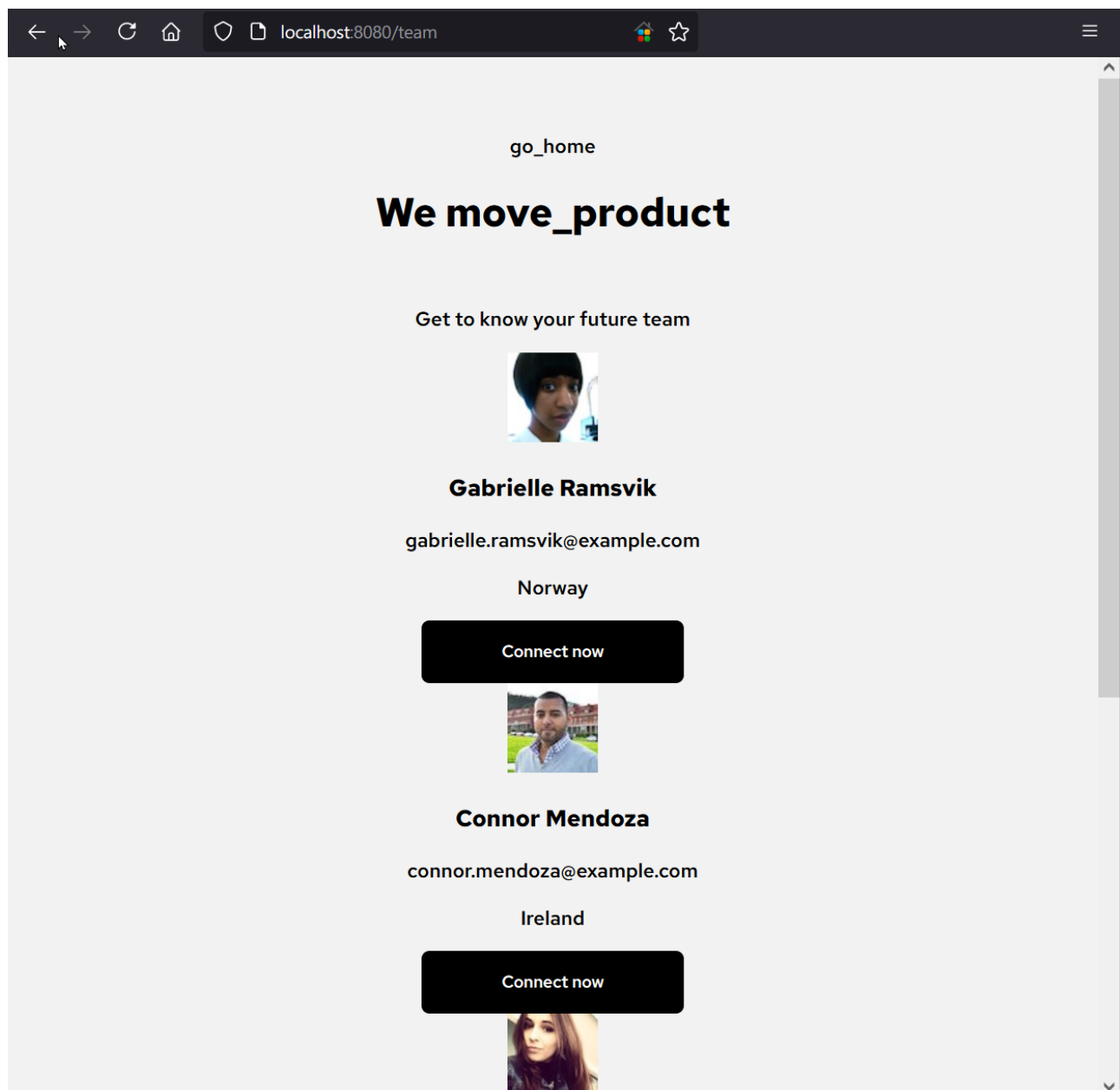
```
</script>
```

Let's update `Team.vue` to call `TeamMemberList` and bind the array of `teamMembers`:

```
<template>
```

```
<section class="team-member-view">
  <p>Get to know your future team</p>
  <TeamMemberList v-bind:teamMembers="teamMembers" />
</template>
```

The unstyled but functional result will look like this:



`App.vue`'s style is applied to all other routes, including `/team`. To finalize the look, add:

Team.vue

```
section.team-member-view {  
  padding-top: 2rem;  
  border-top: 4px dashed #c3c3c3;  
}
```

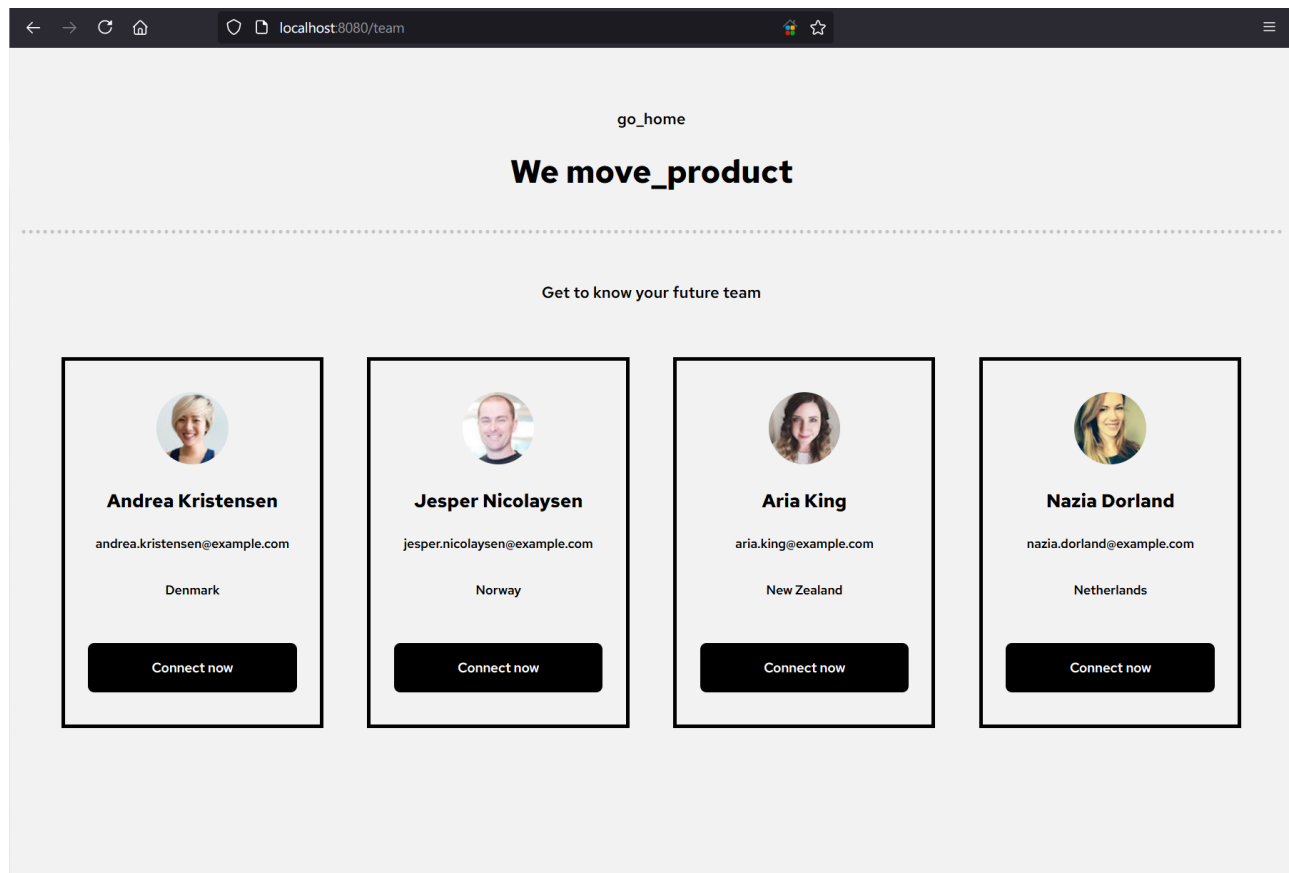
TeamMemberList.vue

```
.team-member-list {  
  margin: 3.4rem auto;  
  display: flex;  
  flex-flow: row wrap;  
  justify-content: space-evenly;  
}
```

TeamMemberCard.vue

```
ul.team-member-card {  
  width: 16rem;  
  display: flex;  
  flex-flow: wrap column;  
  justify-content: space-evenly;  
  border: 4px solid #000000;  
}  
  
ul img {  
  border-radius: 50%;  
  margin-top: 2rem;  
}  
  
ul p, a {  
  padding-top: 1rem;  
  font-size: 0.8rem;  
}  
  
ul button {  
  margin: 2rem 0;  
}
```

Which will yield the result as seen in the beginning of the tutorial:



Congratulations! We've accomplished the following:

- making several views and components,
- routing and rendering them,
- and fetching data.

You now know where certain parts of the logic live, and have an understanding of Vue's file architecture and vocabulary – refs, fetch, router-link, and the like.

Extending your knowledge further

Some suggestions to build on our dynamic landing page:

- The **fetch function** can be improved with a **"loading..." message** and **error handling**
- Explore [watchers and computed properties](#) in Vue 3

Good housekeeping

- Use `PascalCase` for component names (Home vs home, UserTestimonial vs user-testimonial). Readability is improved because we can easily identify components versus

other objects in the code. Notice also that ESLint suggests the use of camelCase in variables within Vue objects. (Subject to agreed upon coding conventions within teams.)

- The structure of `.vue` files (for example, does `<script>` go above or below `<style>?`) and Vue object properties should be standardized for both Options and Composition API, possibly with the user of a boilerplate available through a coding convention document.

Credits

- <https://randomuser.me/> for the open-source, public API

Github Repo

Source: https://github.com/shubhendra-deepsources/move_product